

线性结构：链表 · 栈 · 队列

链表 (Linked List)

核心思想：不需要连续内存。每个节点存数据 + 下一节点的指针。

对比数组：数组 $O(1)$ 查找但 $O(N)$ 插入/删除；链表 $O(N)$ 查找但 $O(1)$ 插入/删除。

节点结构

```
typedef struct node {
    int data;
    struct node *next;
} Node;
```

头部插入：需要改 head 本身，传 Node**

```
void insertHead(Node** head, int x) {
    Node* p = createNode(x);
    p->next = *head;
    *head = p;           // 改main里的head
}
```

创建节点

```
Node* createNode(int x) {
    Node* p = (Node*)malloc(sizeof(Node));
    p->data = x;
    p->next = NULL;    // 必须初始化为NULL!
    return p;
}
```

易错点

插中间时先 `node->next=p` 后 `p->next=node->next` → p 指向自己形成环！

头部插入用 Node* 传参 → 改不了 main 的 head。

删除 (Delete)

删第一个值为 x 的节点。分两情况：删头 vs 删中间。一定要 free！

```
void deleteNode(Node** head, int x) {
    // Case 1: 删头节点
    if ((*head)->data == x) {    // 注意括号!
        Node* tmp = *head;
        *head = (*head)->next;
        free(tmp); return;
    }
    // Case 2: 删中间节点
    Node* p = *head;
    while (p->next && p->next->data != x)
        p = p->next;           // 必须判 p->next 非 NULL
    if (p->next == NULL) return; // 没找到
    Node* tmp = p->next;
    p->next = p->next->next;     // 绕过被删节点
    free(tmp);
}
```

建链：串 3 个节点

```
Node* head = createNode(10);
head->next = createNode(20);
head->next->next = createNode(30);
// 10 -> 20 -> 30 -> NULL
```

遍历 (Traversal)

```
void printList(Node* head) {
    Node* p = head;           // 用临时指针
    while (p != NULL) {
        printf("%d\n", p->data);
        p = p->next;
    }
}
```

易错点

Node* p; 只管声明不管分配 → 野指针。
遍历用 head 直接移动 → head 丢了再也找不到链表头。

插入 (Insert)

在 node 后面插入：注意顺序！先连新节点的 next，再改 node 的 next。

```
void insertAfter(Node* node, int x) {
    Node* p = createNode(x);    // 先建节点
    p->next = node->next;        // 新连后
    node->next = p;              // 前连新
    // 顺序反了链表会断！
}
```

易错点

*head->data 优先级错 → *(head->data)，应为 (*head)->data。

NULL 检查放在 p->next->data 后面 → 先崩了才判断。忘了 free。

x 不存在时 p->next->next 段错误。

双向循环链表

```
typedef struct node {
    struct node *llink; // 指向前一个
    int item;
    struct node *rlink; // 指向后一个
}
```

```
} Node;
```

性质: $\text{ptr} = \text{ptr} \rightarrow \text{llink} \rightarrow \text{rlink} = \text{ptr} \rightarrow \text{rlink} \rightarrow \text{llink}$

栈 (Stack) —LIFO

后进先出。操作: Push(入栈)、Pop(出栈)、Top(查看栈顶)。

数组实现

```
#define MAX 100
typedef struct { int data[MAX]; int top; } Stack;
void init(Stack* s) { s->top = -1; }
void push(Stack* s, int x) {
    if(s->top>=MAX-1)return;
    s->data[++(s->top)] = x; // 先+后放
}
int pop(Stack* s) {
    if(s->top== -1) return -1;
    return s->data[(s->top)--]; // 先取后-
}
int isEmpty(Stack* s) { return s->top== -1; }
```

易错点

push 忘判满, pop 忘判空。
data[s->top++] = x 后自增错 → data[-1]。

多项式 ADT

链表表示: $\langle e_i, a_i \rangle$ 按指数降序。
系数数组浪费空间 (稀疏多项式), 链表节省空间。

栈应用一: 后缀表达式求值

遇到数字 push, 运算符 pop 两个、算完 push 回去。
例: 6 2 / 3 - 4 2 * + = 8。

扫描	操作	栈
6	push 6	[6]
2	push 2	[6,2]
/	pop2,pop6→6/2=3,push	[3]
3	push 3	[3,3]
-	pop3,pop3→3-3=0,push	[0]
4	push 4	[0,4]
2	push 2	[0,4,2]
*	pop2,pop4→4*2=8,push	[0,8]
+	pop8,pop0→0+8=8,push	[8]

栈应用二: 中缀转后缀

运算符	栈内 ISP	栈外 ICP
(	0	6
+ -	3	2
* /	5	4
)	—	1

规则: $\text{ICP} > \text{ISP}$ 入栈, 否则弹栈直到条件满足。

```
void infixToPostfix() {
    Stack s; init(&s); char ch;
    while((ch=getchar())!='\n'){
        if(ch=='\t')continue;
        if(ch>='0'&&ch<='9') printf("%c",ch);
        else if(ch=='(') push(&s,ch);
        else if(ch==''){
            while(top(&s)!='(') printf("%c",pop(&s));
            pop(&s); // 弹掉(
        } else { // + - * /
            if(top(&s) != '(' && isp(top(&s)) >= icp(ch))
                printf("%c",pop(&s));
            push(&s,ch);
        }
    }
}
```

游标实现 (无指针)

全局数组 CursorSpace[], 索引 0 是自由链表头。
malloc: $\text{p} = \text{CursorSpace}[0].\text{Next}; \text{CursorSpace}[0].\text{Next} = \text{CursorSpace}[\text{p}].\text{Next};$
free: $\text{CursorSpace}[\text{p}].\text{Next} = \text{CursorSpace}[0].\text{Next};$
 $\text{CursorSpace}[0].\text{Next} = \text{p};$

```
    }
    while(!isEmpty(&s)) printf("%c",pop(&s));
}
```

易错点

忘了弹掉（。忘了最后弹空栈。
只写 push(ch) 没有 &s。

手算示例：(1+2)*3-4/(5+6)
→ 1 2 + 3 * 4 5 6 + / -

栈应用三：函数调用

每次函数调用创建**栈帧 (Stack Frame)**：返回地址、局部变量、旧帧指针。递归层数太深 → 栈溢出。
尾递归消除：尾递归可转为迭代。例：PrintList 尾递归 → while 循环。

队列 (Queue) —FIFO

先进先出。Enqueue(入队尾)、Dequeue(出队首)。

简单数组队列

```
typedef struct { int data[MAX]; int front,rear; } Queue;
void init(Queue* q) { q->front=0; q->rear=-1; }
void enq(Queue* q,int x) { q->data[++(q->rear)]=x; }
int deq(Queue* q) {
    if(q->front>q->rear)return -1;    // 判空
    return q->data[(q->front)++];
}
```

缺点： front 前面的空间永远浪费。

线性结构复杂度总结

操作	数组	单向链表	双向链表
查找第 k 个	$O(1)$	$O(N)$	$O(N)$
头部插入	$O(N)$	$O(1)$	$O(1)$
尾部插入	$O(1)$ (有 size)	$O(N)$ (无尾针)	$O(1)$
中间插入	$O(N)$	$O(1)$ (已知前驱)	$O(1)$
删除	$O(N)$	$O(1)$ (已知前驱)	$O(1)$

循环队列

数组当环用。front=rear 为空，(rear+1)%MAX==front 为满。
浪费一个格子区分空/满，最多存 MAX-1 个。

```
void init(Queue* q) { q->front=0; q->rear=0; }
void enq(Queue* q, int x) {
    if((q->rear+1)%MAX==q->front) return;    // 先判满
    q->rear=(q->rear+1)%MAX;
    q->data[q->rear]=x;
}
int deq(Queue* q) {
    if(q->front==q->rear)return -1;           // 判空
    int x=q->data[q->front];
    q->front=(q->front+1)%MAX;
    return x;                                // return放最后
}
```

易错点

init 写成 rear=-1（循环队列应为 0）。
先移 rear 再判满 → 万一满的，rear 动了但数据没放。
dequeue 里 return 后还有 front++ → 永远不会执行。
循环队列空用 front>rear → 应 front==rear。

链表实现栈/队列

栈：链表头插头删（头作栈顶）— $O(1)$ 。
队列：链表尾插头删（维护头尾指针）— $O(1)$ 。

层序遍历用队列

二叉树的层序遍历：根入队 → 出队访问 → 左右孩子入队 → 重复。

二叉树 + 二叉搜索树 (BST)

1. 树的术语

术语	定义
Degree(节点的度)	该节点的直接孩子数
Degree of Tree	所有节点度的最大值
Parent/Child/Sibling	父/子/兄弟
Leaf/Terminal	度为 0 的节点 (没有孩子)
Path	节点序列 $n_1, n_2, \dots, n_k$
Path Length	路径上的 边数 (不是节点数)
Depth	从根向下的边数, 根 = 0
Height	到最深叶子的边数, 叶子 = 0
Ancestor	从节点到根的路径上所有节点
Descendant	该节点子树中所有节点

考点

深度 $\neq$ 高度: 深度从上往下, 高度从下往上。

叶子 $\neq$ 孩子: 叶子是度 = 0 的节点, 孩子是有父节点的节点。

路径长度 = 边数 (不是节点数)

3. 四大遍历 (考试必默写)

```
// 先序: 根 → 左 → 右
void preorder(TreeNode* r) {
    if(!r) return;
    printf("%d\n", r->data);
    preorder(r->left);
    preorder(r->right);
}

// 中序: 左 → 根 → 右
void inorder(TreeNode* r) {
    if(!r) return;
    inorder(r->left);
    printf("%d\n", r->data);
    inorder(r->right);
}

// 后序: 左 → 右 → 根
void postorder(TreeNode* r) {
    if(!r) return;
    postorder(r->left);
    postorder(r->right);
    printf("%d\n", r->data);
}
```

关键: 三种递归只差 printf 的位置。

2. 二叉树三性质

1. 第 i 层最多 2^{i-1} 个节点 ($i \geq 1$)

2. 深度 k 的树最多 $2^k - 1$ 个节点

3. $n_0 = n_2 + 1$ (叶子数 = 度 2 节点数 + 1)

性质 3 证明: $n = n_0 + n_1 + n_2, B = n_1 + 2n_2, n = B + 1$ 。

代入得证。

空链接数: n 个节点的二叉树有 $n + 1$ 个 NULL 链接。

中序遍历—迭代版

```
void iterInorder(TreeNode* r) {
    Stack S; init(&S);
    for(;;){ // 也可 while(t!!!isEmpty(&S))
        for(; r; r=r->left) push(&S, r);
        if(isEmpty(&S)) break;
        r=pop(&S); visit(r);
        r=r->right;
    }
}
```

二叉树类型

斜二叉树: 全左或全右 → 最坏深度 $O(N)$ 。

完全二叉树: 除最底层, 其他全满。最底层从左到右连续。

层序遍历—用队列

```
void levelorder(TreeNode* r) {
    if(!r) return; Queue q; init(&q);
    enqueue(&q, r);
    while(!isEmpty(&q)){
        TreeNode* t=dequeue(&q);
        visit(t);
        if(t->left) enqueue(&q, t->left);
        if(t->right) enqueue(&q, t->right);
    }
}
```

表达式树

叶子 = 操作数, 内部节点 = 运算符。

后缀表达式 → 表达式树: 用栈构建。读到操作数 push, 读到运算符 pop 两个当孩子再 push。

三种遍历对应: 中序 → 中缀, 后序 → 后缀, 先序 → 前缀。

遍历结果速记

遍历	顺序	用途
先序 (Pre)	根 → 左 → 右	目录列表
后序 (Post)	左 → 右 → 根	目录大小、表达式求值
中序 (In)	左 → 根 → 右	BST 输出有序序列
层序	BFS	逐层遍历

4. 线索二叉树

动机：n 个节点的二叉树有 n + 1 个 NULL。用来指向中序前驱/后继。

规则：

- 1. Tree 上->Left=NULL → 指向中序前驱
- 2. Tree 上->Right=NULL → 指向中序后继
- 3. 加头节点，无悬空线索

节点结构：data, left, right, leftThread, rightThread

5. 二叉搜索树 (BST)

定义：左子树所有值 < 根 < 右子树所有值。键值互异（或可处理重复）。

Find —O(h)

```
TreeNode* find(TreeNode* r, int x) {
    if(!r) return NULL;
    if(x<r->data) return find(r->left,x);
    if(x>r->data) return find(r->right,x);
    return r;
}
// 迭代版: while(r){if(x<r->data)r=r->left;else if...else return r;}
```

FindMin / FindMax —一直左 / 一直右

```
TreeNode* findMin(TreeNode* r) {
    if(!r)return NULL;
    if(!r->left)return r;
    return findMin(r->left);
}
TreeNode* findMax(TreeNode* r) {
    if(!r)return NULL;
    while(r->right) r=r->right;
    return r;
}
```

Insert —在 NULL 处创建新节点

```
TreeNode* insert(TreeNode* r, int x) {
    if(!r) {
        TreeNode* n=(TreeNode*)malloc(sizeof(TreeNode));
        n->data=x; n->left=n->right=NULL;
    }
```

```
        return n;
    }
    if(x<r->data) r->left=insert(r->left,x);
    else if(x>r->data) r->right=insert(r->right,x);
    return r;
}
```

易错点

r 为 NULL 时直接 r->data=x → 段错误。必须先 malloc。忘了 root->left=insert(...) 来接返回值。

Delete —三情况

```
TreeNode* delete(TreeNode* r, int x) {
    if(!r)return NULL;
    if(x<r->data) r->left=delete(r->left,x);
    else if(x>r->data) r->right=delete(r->right,x);
    else { // 找到了!
        // Case 1+2: 0或1个孩子
        if(!r->left){TreeNode*t=r->right;free(r);return t;}
        if(!r->right){TreeNode*t=r->left;free(r);return t;}
        // Case 3: 两个孩子 → 找替身
        TreeNode* min=findMin(r->right); // 右子树最小
        r->data=min->data;
        r->right=delete(r->right,min->data); // 递归删替身
    }
    return r;
}
```

考点

Case 3 核心：找左子树最大或右子树最小替换值，然后删替身（替身必是 Case 1 或 2）。
不要找整棵树的最大！ 应该找左子树的最大。

易错点

忘了 free。
Case 2 的 left/right 写反（只有一个左孩子却返回了右孩子）。
findMax(r) 找整棵树最大值 → 应 findMax(r->left)。
delete(max,max->data) 参数错误 → 应 r->left=delete(r->left,max->data)。

平均情况分析

BST 的深度取决于插入顺序。
随机插入 → 平均深度 O(log N)；升序/降序插入 → 深度 O(N)（退化为链表）。

懒惰删除

加 flag 标记删除，不真正 free。
好处：不用 malloc 就能重插入。坏处：dead node 太多影响效率。

二叉树操作复杂度

操作	时间复杂度	备注
Find / FindMin / FindMax	$O(h)$	h= 树高
Insert	$O(h)$	在 NULL 处创建
Delete	$O(h)$	Case 3 转 Case 1/2
四种递归遍历	$O(N)$	每个节点访问一次
层序遍历	$O(N)$	用队列
BST 随机插入平均深度	$O(\log N)$	
BST 最坏深度	$O(N)$	有序插入

优先队列：堆 (Heap)

堆的两条性质

1. 结构性性质：完全二叉树。
2. 堆序性质：最小堆—父 $\leq$ 所有孩子；最大堆—父 $\geq$ 所有孩子。

数组存储（索引从 1 开始）

关系	公式	例子
Parent(i)	$\lfloor i/2 \rfloor$	parent(4)=2
Left(i)	$2i$	left(2)=4
Right(i)	$2i + 1$	right(2)=5

data[0] 作 sentinel（存极小值 MinData），自动终止上滤循环。

考点

这三个公式只用于数组实现的堆！不适用于 BST、链表等其他结构。

注意区分：原地堆排序用索引从 0 开始（left=2i+1, right=2i+2）。

5 种实现的复杂度对比

实现	Insert	DeleteMin
无序数组	$\Theta(1)$	$\Theta(N)$
无序链表	$\Theta(1)$	$\Theta(N)$
有序数组	$O(N)$	$\Theta(1)$
有序链表	$O(N)$	$\Theta(1)$
BST(平衡)	$O(\log N)$	$O(\log N)$
二叉堆	$O(\log N)$	$O(\log N)$

```
h->data[i] = h->data[i/2];    // 父下沉
i = i/2;                        // i上移
}
h->data[i] = x;                  // 放好
}
// T(N) = O(log N)
```

手算示例：初始堆 [3,6,5,9,10]，Insert(2)。

2 放 data[6] $\rightarrow$ 跟 data[3]=5 比，5>2，5 下沉到 6，i=3。
跟 data[1]=3 比，3>2，3 下沉到 3，i=1。到根，2 放 data[1]。
结果：[2,6,3,9,10,5]。

初始化

```
PriorityQueue Init(int Max) {
    PriorityQueue H = malloc(sizeof(HeapStruct));
    H->Elements = malloc((Max+1)*sizeof(ElementType));
    H->Capacity = Max; H->Size = 0;
    H->Elements[0] = MinData; // sentinel
    return H;
}
```

易错点

int i = h->size++ 后自增拿的是旧值。必须 ++(h->size)。
循环里写成 data[i/2]=x 做了交换 $\rightarrow$ 应该只下沉，最后才放。
忘了循环外的 h->data[i]=x。

Insert — 上滤 (Percolate Up)

思想：新元素放在末尾（保持完全二叉树），然后和父节点比，小的往上冒。

```
void insert(Heap* h, int x) {
    if(h->size >= MAX-1) return;
    int i = ++(h->size); // 先+后取！
    // 父节点比x大就下沉，空穴上移
    while(i>1 && h->data[i/2] > x) {
```

DeleteMin — 下滤 (Percolate Down)

思想：取走根（最小），最后元素补到根，然后跟较小孩子比，大的往下沉。

```
int deleteMin(Heap* h) {
    if(isEmpty(h)) return -1;
    int min = h->data[1]; // 最小值
    int x = h->data[h->size--]; // 最后一个元素
    int i=1, child;
    while(2*i <= h->size) { // 有左孩子
        child = 2*i;
```

```

// 如果右孩子存在且更小，选右
if(child < h->size && h->data[child+1] < h->data[
    child])
    child++;
if(x <= h->data[child]) break;    // 放好了
h->data[i] = h->data[child];      // 孩子上移
i = child;
}
h->data[i] = x;                  // 放好
return min;
}
// T(N) = O(log N)

```

手算示例：堆 [2,6,3,9,10,5]，DeleteMin。

取走 2，x=5。i=1，child=2(左) vs 3(右)，3 更小。5>3，3 上移到 1，i=3。

i=3，child=6(左)，2*i=6 > size=5，无孩子，停。5 放在 3。结果：[3,6,5,9,10]。

其他堆操作

- **DecreaseKey(P, Δ , H)**：减小某键值 $\rightarrow$ 上滤。提高进程优先级。
- **IncreaseKey(P, Δ , H)**：增大某键值 $\rightarrow$ 下滤。降低 CPU 占用。
- **Delete(P, H)**：DecreaseKey(P, ∞ , H) 使其到根 + DeleteMin(H)。
- **BuildHeap(H)**：Floyd 自底向上建堆 $O(N)$ 。

BuildHeap —Floyd 法 $O(N)$

思想：从最后一个非叶节点 ($N/2$) 开始，往前逐个下滤。叶子不用处理。

```

void percdDown(Heap* h, int i) {
    int x = h->data[i], child;
    while(2*i <= h->size) {
        child = 2*i;
        if(child < h->size && h->data[child+1] < h->data[
            child])
            child++;
        if(x <= h->data[child]) break;
        h->data[i] = h->data[child];
        i = child;
    }
    h->data[i] = x;
}

void buildHeap(Heap* h, int arr[], int N) {
    for(int i=0; i<N; i++) h->data[i+1] = arr[i];
    h->size = N;
    for(int j = N/2; j >= 1; j--)          // 从最后一个非叶
        to 根
        percdDown(h, j);
}
// T(N) = O(N) — 不是 O(N log N)!

```

考点

为什么 BuildHeap 是 $O(N)$ 不是 $O(N \log N)$?

定理：完美二叉树中所有节点高度和 = $2^{h+1}-1-(h+1) \approx N$ 。

大部分节点在底层 (叶子) 几乎不下沉，总工作量接近 N 而非 $N \log N$ 。

手算示例：arr=[4,2,7,1,3,6,5]，N=7。N/2=3。

下滤 data[3]=7：7 和 5 换 $\rightarrow$ [4,2,5,1,3,6,7]。

下滤 data[2]=2：跟孩子 1 和 3 比，1 < 2 $\rightarrow$ 2 和 1 换 $\rightarrow$ [4,1,5,2,3,6,7]。

下滤 data[1]=4：跟 1 和 5 比，1 < 4 $\rightarrow$ 4 和 1 换 $\rightarrow$ [1,4,5,2,3,6,7]。再跟 2 和 3 比，2 < 4 $\rightarrow$ 4 和 2 换 $\rightarrow$ [1,2,5,4,3,6,7]。

易错点

for(j=N/2; j>1; j--) 漏了根 j=1。应为 j>=1。

拷数组写了 arr+i (指针地址) $\rightarrow$ 应 arr[i]。

h->size 用 for 循环 ++ $\rightarrow$ 直接 h->size=N。

Floyd 写成 while(j!=1)+j/=2+percdDown 两下 $\rightarrow$ 应是简单 for。

易错点

大小比较反了：最小堆找较小孩子，最大堆找较大孩子。
child!=h->size 条件漏了 $\rightarrow$ 可能访问越界。
忘了最后 h->data[i]=x。

d-堆

- DeleteMin 比较次数 $d - 1$ 增多

每个节点有 d 个孩子。DeleteMin 需 $d-1$ 次比较选最小 $\rightarrow O(d \log_d N)$ 。

索引关系： $\text{parent} = \lfloor (i + d - 2) / d \rfloor$,
 $\text{children} = d(i - 1) + 2, d(i - 1) + 3, \dots, di + 1$ 。

- 大 d 的优缺点：
- + 树更矮，外存场景（磁盘 I/O 少）
 - $*d / /d$ 不是位运算（对比 $*2 / /2$ 是移位）

应用：找第 k 大元素

方法	复杂度
排序取第 k	$O(N \log N)$
大小 k 的最小堆	$O(N \log k)$
BuildHeap + k 次 DeleteMax	$O(N + k \log N)$
QuickSelect	平均 $O(N)$ ，最坏 $O(N^2)$

FDS 排序算法—考前速查

考点

比较排序下界：任意基于比较的排序最坏 $\Omega(N \log N)$ 。
桶排/基数排不基于比较，能突破下界。决策树证明： $N!$ 个叶子，深度至少 $\log N! = \Omega(N \log N)$ 。

算法	平均	最坏	额外空间	稳定?	比较?
插入 (Insertion)	$O(N^2)$	$O(N^2)$	$O(1)$	是	是
希尔 (Shell)	$O(N^{3/2})$	$O(N^2)$	$O(1)$	否	是
堆排 (Heap)	$O(N \log N)$	$O(N \log N)$	$O(1)$	否	是
归并 (Merge)	$O(N \log N)$	$O(N \log N)$	$O(N)$	是	是
快排 (Quick)	$O(N \log N)$	$O(N^2)$	$O(\log N)$	否	是
桶排 (Bucket)	$O(M + N)$	—	$O(M)$	是	否
基数 (Radix)	$O(P(N + B))$	$O(P(N + B))$	$O(N + B)$	是	否

1. 插入排序 (Insertion)

思想：像整理扑克牌，每次从后面取一张插入前面已排好的位置。

核心：不是交换，是右移 + 插入。

```
void insertionSort(int A[], int N) {
    int i, j, tmp;
    for (i = 1; i < N; i++) {
        tmp = A[i];
        for (j = i; j > 0 && A[j-1] > tmp; j--)
            A[j] = A[j-1]; // 比 tmp 大的右移
        A[j] = tmp; // 插入
    }
}
```

复杂度：最好 $O(N)$ (已有序)/最坏 $O(N^2)$ (逆序)。
逆序数：平均 $N(N-1)/4$ 。每次相邻交换消除恰好 1 个逆序。

易错点

tmp=A[i] 后从头开始找位置，而不是从 i 往前比。
for(int j=i;...) 声明的 j 在外面 A[j]=tmp 用不了。

2. 希尔排序 (Shellsort)

思想：不等距的插入排序。gap 从 $N/2$ 逐步缩到 1。

```
void shellSort(int A[], int N) {
    int gap, i, j, tmp;
    for (gap = N/2; gap > 0; gap /= 2)
        for (i = gap; i < N; i++) {
            tmp = A[i];
            for (j=i; j>=gap && A[j-gap]>tmp; j-=gap)
                A[j] = A[j-gap];
            A[j] = tmp;
        }
}
```

跟插入的区别 (只改 3 处)：(1) 外套 gap 循环 (2) j-1 $\rightarrow$ j-gap (3) j-- $\rightarrow$ j-=gap

Hibbard 增量： $h_k = 2^k - 1$ ，最坏 $\Theta(N^{3/2})$ 。
Sedgewick 最优： $\{1, 5, 19, 41, 109, \dots\}$ 。

易错点

while(gap!=1) 吞了 gap=1 这轮。i+=gap 只处理一组！
for(int j=i...) j 作用域只在 for 里，外面用不了。

3. 堆排序 (Heapsort)

思想：原地最大堆。根 (max) 和末尾交换，堆缩小，下滤，重复。

索引从 0 开始！left=2i+1, right=2i+2, parent=(i-1)/2。

```
void percdDown(int A[], int i, int N) {
    int child, tmp = A[i];
    while (2*i+1 < N) {
        child = 2*i + 1;
        if (child+1<N && A[child+1]>A[child]) child++;
        if (tmp >= A[child]) break;
        A[i] = A[child]; i = child;
    }
    A[i] = tmp;
}

void heapSort(int A[], int N) {
    for (int i = N/2-1; i >= 0; i--) // BuildMaxHeap O(N)
        percdDown(A, i, N);
    for (int i = N-1; i > 0; i--) { // 排序 O(N log N)
        int t=A[0]; A[0]=A[i]; A[i]=t;
        percdDown(A, 0, i);
    }
}
```

$T(N) = O(N \log N)$ ，原地，不稳定。

易错点

用最小堆做原地排序 (应该用最大堆)；忘了索引从 0 开始。j>1 漏了根；i<N 漏了最后一个元素。

4. 归并排序 (Mergesort)

思想：分治法。递归拆分，再合并两个有序段。

```
void merge(int A[], int l, int m, int r) {
    int i=l, j=m+1, k=0, n=r-l+1;
    int* T=(int*)malloc(n*sizeof(int));
    while(i<=m && j<=r)
        T[k++]=(A[i]<=A[j])?A[i++]:A[j++];
    while(i<=m) T[k++]=A[i++];
    while(j<=r) T[k++]=A[j++];
    for(int p=0;p<n;p++) A[l+p]=T[p];
    free(T);
}

void mergeSort(int A[], int l, int r) {
    if (l>=r) return;
    int m=(l+r)/2;
    mergeSort(A,l,m); mergeSort(A,m+1,r);
    merge(A,l,m,r);
}
```

$T(N) = 2T(N/2) + O(N) \rightarrow O(N \log N)$ 。需要额外 $O(N)$ 空间。稳定。

易错点

tmp 的 k 从 left 开始（应为 0）；忘了 free；n 声明在 malloc 后面。

5. 快速排序 (Quicksort)

思想：选 pivot，小的放左，大的放右，递归。

最关键的考试排序！

```
int partition(int A[], int l, int r) {
    int pivot = A[r], i = l;
    for (int j=l; j<r; j++)
        if (A[j] < pivot) {
            int t=A[i]; A[i]=A[j]; A[j]=t; i++;
        }
    int t=A[i]; A[i]=A[r]; A[r]=t;
    return i;
}

void quickSort(int A[], int l, int r) {
    if (l>=r) return;
    int p=partition(A,l,r);
    quickSort(A,l,p-1); quickSort(A,p+1,r);
}
```

手算：[5,2,6,1,4,9,3,7], pivot=7。i 是小区域的边界，j 扫描。遇到比 7 小的就换到 i 处。最后 pivot 放 i 处。
结果：[5,2,6,1,4,3,7,9]，pivot=7 位置 6。

Median-of-Three 优化：取 $A[l]$, $A[(l+r)/2]$, $A[r]$ 的中值作 pivot。

最坏：每次 pivot 最小/最大 $\rightarrow O(N^2)$ ；**平均：** $O(N \log N)$ 。
不稳定。

易错点

pivot 选 $A[0]$ 遇到已排序数组直接退化 $O(N^2)$ ；忘了 swap pivot 到中间。

6. 表排序 (Table Sort)

动机：大结构体移动代价高。只排索引表 T。

$A = [f, d, a, b, e, c]$

$T = [0, 1, 2, 3, 4, 5] \rightarrow$ 排序后 $T = [2, 3, 5, 1, 4, 0]$
 $T[0]=2 \rightarrow A[2]=a$ (最小)

物理重排 (置换环)：把 A 按 T 重排。每个环移动 (环长 +1) 次。

最坏 $\lfloor N/2 \rfloor$ 个环，共 $\lfloor 3N/2 \rfloor$ 次移动。 $T = O(mN)$, $m =$ 结构体大小。

```
void rearrange(ElementType A[], int T[], int N) {
    for (int i=0; i<N; i++)
        if (T[i]!=i) {
            ElementType tmp=A[i]; int j=i;
            while(T[j]!=i){
                A[j]=A[T[j]]; int next=T[j];
                T[j]=j; j=next;
            }
            A[j]=tmp; T[j]=j;
        }
}
```

7. 桶排序 (Bucket Sort)

思想：不比较，按值分类投放。每个桶就是一个计数器。

```
// 例：给 N 个 O-M 分的学生成绩排序
void bucketSort(int A[], int N, int M) {
    int* count = (int*)calloc(M+1, sizeof(int));
    for (int i=0; i<N; i++) count[A[i]]++;
    for (int i=0, j=0; i<=M; i++)
        while (count[i]-- > 0) A[j++] = i;
    free(count);
}
```

$T = O(M + N)$ 。M = 桶数。**不基于比较**，突破 $O(N \log N)$ 。稳定。

8. 基数排序 (Radix Sort —LSD)

思想：从最低位 (LSD) 到最高位 (MSD)，每轮按当前位入桶排序。

手算：[64,8,216,512,27,729,0,1,343,125]

Pass1(个位): 0:[0] 1:[1] 2:[512] 3:[343] 4:[64] 5:[125]
6:[216] 7:[27] 8:[8] 9:[729]
 $\rightarrow 0, 1, 512, 343, 64, 125, 216, 27, 8, 729$

Pass2(十位): 0:[0,1,8] 1:[512,216] 2:[125,27,729]
3:[343] 6:[64]
 $\rightarrow 0, 1, 8, 512, 216, 125, 27, 729, 343, 64$

Pass3(百位): 1:[125] 2:[216] 3:[343] 5:[512] 7:[729]
0:[0,1,8,27,64]
 $\rightarrow 0, 1, 8, 27, 64, 125, 216, 343, 512, 729$

```
// LSD Radix Sort, 假设 10 进制, maxDigits=最大位数
void radixSort(int A[], int N, int maxDigits) {
    int cnt[10], exp = 1;
    int tmp[N];
    for (int d = 0; d < maxDigits; d++) {
        for (int i = 0; i < 10; i++) cnt[i] = 0;
        for (int i = 0; i < N; i++)
            cnt[(A[i] / exp) % 10]++;
        // 前缀和 → 各桶起始位置
        for (int i = 1; i < 10; i++) cnt[i] += cnt[i - 1];
        // 从后往前收集 (保证稳定!)
        for (int i = N - 1; i >= 0; i--) {
```

```
int digit = (A[i] / exp) % 10;
tmp[--cnt[digit]] = A[i];
}
for (int i = 0; i < N; i++) A[i] = tmp[i];
exp *= 10;
}
```

$T = O(P \cdot (N + B))$, P= 位数, B= 基数 (10)。稳定, 不基于比较。

MSD vs LSD: MSD 从高到低 (分治), LSD 从低到高 (稳定)。

卡片排序: LSD 从最不重要到最重要字段依次排。

决策树与排序下界

考点

决策树证明 $\Omega(N \log N)$ 下界

- N 个元素有 $N!$ 种排列 $\rightarrow$ 决策树至少 $N!$ 个叶子
- 二叉树深度 k 最多 2^{k-1} 个叶子
- $N! \leq 2^{k-1} \rightarrow k \geq \log N! + 1 = \Omega(N \log N)$

3 个元素插入排序决策树:

$N = 3, N! = 6$, 深度 $k \geq 4$ (至少 4 次比较)。

稳定性总结

稳定: 插入、归并、桶、基数。

不稳定: 希尔、堆、快排。

口诀: 稳定排队 (插入归并桶基数), 不稳定快跑 (快希尔堆)。

Additional: Replacement Selection

外部排序的预处理: 用最小堆生成更长的初始顺串 (run)。

平均 run 长度 $= 2M$ (M= 堆大小), 比普通分块长一倍。

哈希表 (Hashing) — 考前速查

考点

核心突破：不通过比较查找， $O(1)$ 定位。
基于比较的查找下界 $\Omega(\log N)$ ，哈希突破了它。

1. 基本概念

b (buckets)	桶数量
s (slots/bucket)	每桶槽位数
n	当前元素数
T	所有可能 key 总数
$\lambda = \frac{n}{s \cdot b}$	负载密度

```
Node* p = table[key % T];
while (p && p->key != key) p = p->next;
return p;
}
```

缺点：需要额外指针空间，缓存不友好（内存不连续）。

冲突 (Collision)： $f(i_1) = f(i_2)$ ，两不同 key 映射到同桶。

溢出 (Overflow)：桶已满又来了新 key。

当 $s = 1$ 时，冲突 = 溢出。

例子：10 个 C 函数， $b=26, s=2, \lambda = 10/52 = 0.19$ 。
 $\text{hash}(x)$ = 首字母-'a'。acos/atan 都在桶 0。平均查找 = 3.73。

2. 哈希函数

要求：计算快 + 分布均匀（无偏）。
整数：

```
h(x) = x % TableSize
```

TableSize 必须质数！ 否则后几位相同者扎堆。

字符串—Hash3 (推荐)：

```
int hash(const char* key, int T) {
    unsigned int h = 0;
    while (*key) h=(h<<5)+*key++; // h*32+下一字符
    return h % T;
}
```

用 32 因为 $32 = 2^5$ ，移位比 $*27$ 快。
长字符串：可精选部分字符，早期字符被左移出去丢失影响。

3. 分离链接法

每桶挂一个链表。冲突的串在一起。

```
typedef struct Node { int key; struct Node* next; } Node;
Node* table[T];

void insert(int key) {
    int h = key % T;
    Node* node = malloc(sizeof(Node));
    node->key = key;
    node->next = table[h]; // 头插法 O(1)
    table[h] = node;
}

Node* find(int key) {
```

4. 开放地址法

冲突后按探测序列找下一个空位：
 $h_i = (hash(key) + f(i)) \bmod TableSize$

线性探测 $f(i) = i$

每次 +1。一次聚集： hash 相同的人扎堆。
期望探测次数：插入失败 $\frac{1}{2}(1 + 1/(1 - \lambda)^2)$

二次探测 $f(i) = i^2$

+1,+4,+9,+16...
定理：若 TableSize 是质数且表 > 1/2 空，一定找到空位。
(前 T/2 个 $i^2 \bmod$ 质数互不相同)

二次聚集：同 hash 者探测路径一样。
代码：

```
int find(int T[], int key, int size) {
    int h = key % size;
    for (int i=0; i<=size/2; i++) {
        int p = (h + i*i) % size;
        if (T[p]==EMPTY) return -1;
        if (T[p]==key) return p;
    }
    return -1;
}
```

插入优化公式： $f(i) = f(i - 1) + 2i - 1$ (避免每次算 i^2)。
懒惰删除：加 flag 标记删除，不真正清空。

双哈希 $f(i) = i \cdot h_2(key)$

推荐： $h_2(x) = R - (x \% R)$, R 为小于 TableSize 的质数 (如 7)。
步长取决于第二个哈希函数，基本消除聚集。

三种探测对比

	聚集	性能
线性	一次聚集严重	最差
二次	二次聚集	中等
双哈希	基本无	最好

5. 再哈希 (Rehashing)

何时触发：
• 插入失败 (没空位)
• λ 超阈值 (如 > 0.5)
• 查找性能严重下降

步骤：
1. 建更大的表 (约 2 倍，质数)
2. 旧表全部元素重哈希插入新表
3. 释放旧表
 $O(N)$ 一次，但摊还到每次插入 $O(1)$ 。
交互系统注意：不能一次性停太久——可逐步迁移。

6. 分离链接 vs 开放地址

	分离链接	开放地址
实现	链表	纯数组
空间	额外指针	无额外开销
满后	无上限	Rehashing
λ	可 >1	必须 <1
缓存	差	好 (连续)

7. 线性探测反推输入顺序

考点

给定哈希表状态，反推插入顺序。线性探测下，元素从 hash 到实际位置之间经过的所有人都比它先插入。

入度公式： $indeg[i] = (i - hash(a[i]) + N) \% N$

然后拓扑排序：每次选入度为 0 且值最小的输出。

例子： N=5, 表 =[10,6,15,0,8]。
 $a[4]=8, h=3, in=1$ (依赖 3); $a[3]=0, h=0, in=3$ (依赖 0,1,2)...
最小候选优先输出。

并查集 + 图论—考前速查

并查集 (Union-Find / Disjoint Set)

问题： N 个元素，支持 Find(查集合) 和 Union(合集合)。
应用：等价关系、Kruskal MST、连通分量。

数据结构

```
int S[MAX]; // S[root] = -size (负数表示大小)
           // S[x] = parent (正数表示父节点索引)
```

初始化：每个元素自成一个集合

```
void init(int N) { for(int i=0;i<N;i++) S[i]=-1; }
```

Find 一路径压缩

```
int find(int x) {
    if (S[x] < 0) return x; // 是根
    return S[x] = find(S[x]); // 爹变祖宗!
}
```

效果：查找路径上所有节点直连根，后续近乎 $O(1)$ 。

四个版本对比

版本	操作	复杂度
朴素	随便挂	$\Theta(N^2)$ 最坏
US	小挂大	$O(N + M \log N)$
PC	爹变祖宗	每操作均摊接近常数
US+PC	两者都有	$O(M\alpha(M, N))$

考点

α 是反 Ackermann 函数， $\alpha \leq 4$ 在实际数据内。
Union-by-Size + 路径压缩 = 几乎常数时间！
路径压缩不能和 Union-by-Height 一起用（压缩改变高度），改用 Union-by-Rank。

Union 一按大小合并

```
void un(int a, int b) {
    a=find(a); b=find(b);
    if(a==b) return;
    if(S[a]>S[b]){int t=a;a=b;b=t;} // a更大
    S[a]+=S[b]; // 更新大小
    S[b]=a; // b挂到a
}
```

树高： $\leq \lfloor \log_2 N \rfloor + 1$ 。

$\log^* N$ (迭代对数)

$\log^* 65536 = 4, \log^* 2^{65536} = 5$ 。计算公式：反复取 $\log$ 到 ≤ 1 的次数。

Ackermann 函数

$A(0, j) = 1, A(1, 0) = 2, A(i, 0) = i + 2 (i \geq 2), A(i, j) = A(i - 1, A(i, j - 1))$ (双递归，极快增长)。 α 是其反函数——极慢增长。

图论 (Graph)

基本概念

$G = (V, E)$ 。无向图： $(v_i, v_j) = (v_j, v_i)$ 。有向图： $\langle v_i, v_j \rangle \neq \langle v_j, v_i \rangle$ 。
DAG：有向无环图。**握手定理**： $\sum deg(v) = 2|E|$ 。